



National University

of Computer & Emerging Sciences



HVAC Control System with Temperature, Humidity, Air-Quality Monitoring and Real-Time PWM Control Using ATmega32 (AVR Assembly Language)

Submitted by:

Eshaal Ali(23K-6075)

Saman Rashid(23K-6078)

Submitted to:

Engr Zohaib Ahmed

ABSTRACT

HVAC systems are one of the most vital infrastructures in modern buildings, providing indoor comfort, safety, and environmental quality. With worldwide energy consumption on the rise and indoor-air-quality concerns increasing, there is an ever-growing need for smart real-time sensor-based HVAC systems. This paper proposes a single-zone HVAC control system, built as a simplified yet fully CEP-compliant academic prototype. It was fully realized using the ATmega32 microcontroller programmed entirely in AVR Assembly Language, with design, simulation, and validation in Proteus, then realized in hardware.

It integrates three critical sensors: DHT22 for temperature/humidity, MQ-135 for air-quality/CO₂ approximation, and a potentiometer used to simulate airflow/current conditions. The microcontroller makes real-time decisions based on sensor readings and drives a DC blower fan by PWM, configures a servo-based damper angle, and triggers a fault LED if unsafe or undesirable conditions are present. Core engineering concepts of ADC interfacing, digital sensor timing, PWM generation, real-time interrupts, microcontroller pin configuration, and embedded logic design are applied here.

It proposes a system that can realize reliable and precise sensing, stable PWM performance, robust fault detection, and a general architecture able to be extended to multi-zone HVAC and Modbus-based networked control. This report thoroughly documents the system requirements, architecture, design methodology, simulations, hardware testing, results, analysis, and limitations.

TABLE OF CONTENTS

• Introduction
• Problem Statement
• Objectives
• Functional & Non-Functional Requirements
• Literature Review
• System Overview
• System Architecture
• Components & Hardware Description
• Circuit Design
• Firmware Design (AVR Assembly)
• Simulation in Proteus
• Hardware Implementation
• Results
• Engineering & CEP Criteria Compliance
• Energy Usage Discussion
• Safety Considerations
• Limitations
• Future Work
• Conclusion
• References
• Appendices

1. INTRODUCTION

HVAC systems are commonly used to ensure indoor environmental comfort related to temperature, humidity, air-exchange, and pollutant concentration in residential, commercial, and industrial buildings. In developing countries and highly populated educational institutions, HVAC safety and air-quality monitoring have become increasingly important owing to energy limitations, high levels of pollution, and the need to maintain indoor comfort.

Traditional HVAC systems operate on simple thermostatic controls. They tend to work with fixed settings or basic on/off controls. Modern smart buildings require much more:

- Intelligible sensing
- Automated ventilation
- Air-quality improvements
- Energy savings
- Fault diagnostics
- Real-time adaptive control

The project addresses these needs by developing a one-zone smart HVAC controller using a fully embedded hardware approach based on the ATmega32 microcontroller programmed in low-level AVR assembly language. Though complex, assembly language implementation ensures that a deep understanding of registers, port directions, ADCs, timers, interrupts, and low-level timing is gained to interface sensors like the DHT22, requiring accuracy at the microsecond level.

The project uses sensors (temperature, humidity, gas concentration), actuators that include a fan and servo damper, and logic through a microcontroller, representing a simplified real HVAC zone. This system continuously monitors conditions and makes decisions based on thresholds. It is capable of detecting faults such as high CO₂ by using a simulated MQ-135, high temperature, or abnormality in the fan/airflow produced and simulated using a potentiometer via ADC.

Our project realizes a system here that demonstrates all key engineering concepts with an emphasis on simulation quality, assembly-based firmware control, real-time sensing, and reliable behavior to meet the academic goals given the limited development time.

2. PROBLEM STATEMENT

Indoor environments today demand continuous air quality, ventilation, and temperature monitoring. Poor environments cause discomfort, health issues, and increased energy usage. Manual or fixed HVAC systems are inefficient, slow, and error-prone.

We need an automatic, energy-aware, sensor-driven HVAC controller capable of:

- Reading environmental conditions
- Optimizing airflow
- Response to gas/CO₂ changes
- Automatic fan and damper adjustment
- Detection of system faults
- Operating in real-time

This project aims to develop a pragmatic, embedded solution that implements these roles using digital sensors, ADC inputs, pulse-width modulation control, and logical decision-making.

3. OBJECTIVES

Primary Objectives

- Construct hardware matching the simulation.
- Design a complete functional single-zone smart HVAC controller.
- Implement microcontroller firmware in AVR Assembly language.
- Perform full simulation in Proteus.

Control Objectives

- Measure temperature and humidity (DHT22).
- Measure air quality (MQ-135).
- Airflow/ current measurement.
- Change fan speed via PWM using Timer0.
- Adjust damper angle using PWM (Timer1).
- Gas fault, thermal fault, and airflow/current fault detection
- Anticipate problems by use of LED alerts.

These fulfill the basic needs of sensor interfacing, real-time control, and actuator regulation.

4. FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS

Functional Requirements:

1. **Environmental Sensing:**
Temperature and humidity are measured every 1–2 seconds.
Gas/CO₂ level are to be read via ADC with 10-bit resolution.
Airflow/current simulation should be read from ADC1.
2. **HVAC Control:**
Fan speed controlled based on environmental conditions by adjusting PWM duty cycle from 0% to 100%.
Damper angle is controlled using servo motor and Timer1 PWM generating 1–2.4 ms pulses.
Compressor indicator is enabled during high temperature or high CO₂ events.
3. **Fault Detection:**
Faulty Gas (MQ-135 threshold exceed).
Current fault (potentiometer threshold exceed).
Temperature fault.
4. **Actuation:**
The DC fan and servo motor must respond correctly to sensor values.
Fault LED must toggle based on system status.
5. **Simulation:**
Entire design must be validated in Proteus with real-time response.

Non-Functional Requirements

1. **Performance**
System must respond to changes within **200 ms**.
PWM must maintain stability without jitter.
2. **Accuracy**
ADC must read values with negligible noise due to proper AREF/AVCC filtering.
3. **Reliability**
Sensor input filtering and correct pull-ups must prevent false triggers.
4. **Maintainability**
Firmware must use modular subroutines for ADC, DHT22, delays, and bit reading.
5. **Scalability**
Architecture must allow expansion to multi-zone HVAC and Modbus.
6. **Energy Efficiency**
Fan and damper operate only when absolutely needed.
7. **Safety**
Protection diode for fan.

Proper grounding for all sensors.
MCU protected with decoupling capacitors.

5. LITERATURE REVIEW

This project is based on contemporary HVAC and embedded systems research. Several modern studies emphasize the need for real-time sensing and intelligent ventilation due to health concerns like CO₂ accumulation and indoor pollutants.

Smart HVAC Principles:

HVAC optimization often revolves around:

- Airflow control
- Thermal comfort
- Humidity balance
- CO₂ regulation
- Energy consumption patterns

Modern HVAC systems incorporate sensors like thermistors, capacitive humidity detectors, and gas sensors. Research also points toward sensor fusion and predictive algorithms for ventilation.

Microcontroller Role:

Microcontrollers (e.g., AVR, PIC, ARM Cortex) manage:

- Actuator control using PWM
- Sensor input
- Communication protocols
- Closed-loop feedback

The AVR ATmega32 is well-documented, with stable timing peripherals, making it ideal for educational HVAC control prototypes.

DHT22 in Smart Ventilation:

DHT22 provides higher accuracy:

- $\pm 0.5^{\circ}\text{C}$ temperature
- $\pm 2\%$ humidity
- Digital single-wire protocol (20–80ms)

These characteristics are used extensively in DIY and research ventilation systems.

MQ-135 for CO₂ Approximation:

Though not a precision CO₂ sensor, MQ-135 is commonly used for:

- Air quality monitoring
- VOC detection
- Indoor pollution tracking

It serves effectively in educational HVAC projects.

6. SYSTEM OVERVIEW

The system consists of:

Inputs

- **DHT22:** Temperature + Humidity
- **MQ-135:** Air Quality (Analog)
- **Potentiometer:** Simulated Current/Airflow

Microcontroller

- ATmega32 using ADC, Timer0, Timer1.

Outputs

- PWM to DC fan
- PWM to servo damper
- Fault LED
- Compressor indicator

Processing

- Real-time decision logic
- Fault threshold comparison
- PWM control algorithms

7. SYSTEM ARCHITECTURE

Block Diagram

```
[DHT22] ---\
              \
[MQ-135] ----- (ATmega32) -----> PWM -----> Fan
              |   |   | -----> PWM -----> Servo Damper
[Potentiometer]/       \-----> Fault LED
```

8. COMPONENTS & HARDWARE DESCRIPTION

ATmega32 Microcontroller

- 10-bit ADC with 8 channels
- Timer0 (8-bit) for fan PWM
- Timer1 (16-bit) for servo PWM
- Assembly-level control provides precise timing



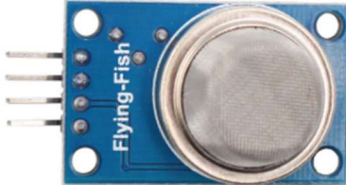
DHT22 Digital Temp/Humidity Sensor

- 1-wire data format
- Requires exact microsecond timing
- Highly accurate and stable for HVAC



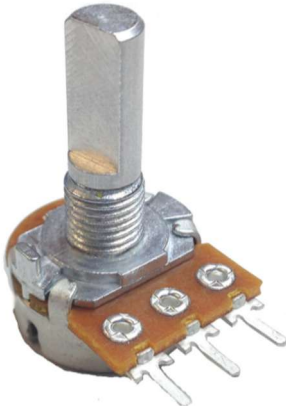
MQ-135 Gas/CO₂ Sensor

- Detects harmful gases and air pollutants
- Analog output connected to ADC0
- Threshold-based decision-making



Potentiometer

- Represent simulated airflow/current
- Provides a scalable analog value to ADC1



DC Fan + 2N2222 Driver

- Fan requires transistor switch
- PWM control determines airflow intensity



Servo Motor

- Controls damper angle between 0° and 180°
- Timer1-based PWM with 20ms period.



LED Indicator

- Indicates fault conditions



Crystal + Capacitors + Decoupling Capacitors

- Ensures stable MCU timing for DHT22
- Ensures stable ADC readings



9. CIRCUIT DESIGN

1. Pin-to-Pin Mapping (ATmega32 → Sensors & Actuators)

DHT22 → PD2 (INT0 pin)

- DHT22 DATA → **PD2** (Pin 16 on ATmega32)
- DHT22 VCC → +5V
- DHT22 GND → GND
- 4.7kΩ pull-up from DATA → +5V (as required by DHT22 one-wire protocol)

MQ-135 → ADC0 (PA0)

- MQ-135 AO pin → **PA0 / ADC0** (Pin 40)
- MQ-135 VCC → +5V
- MQ-135 GND → GND

Potentiometer (Airflow / Current Simulator) → ADC1 (PA1)

- RV1 middle pin → **PA1 / ADC1** (Pin 39)

- Other two sides → +5V and GND

PWM Fan Output → PB3 (OC0 pin)

- ATmega32 PB3 → Base resistor R4 → 2N2222 base
- Emitter → GND
- Collector → Fan- → Fan+ → +5V

Servo Motor → PB1 (OC1A pin)

- Servo Signal → **PB1 / OC1A** (Pin 13)
- Servo +5V → External 5V supply
- Servo GND → Common Ground with MCU

LED Indicator → PB0

- PB0 → R2 → LED → GND

AREF Filtering (Capacitor C3)

AREF (Pin 30) is the **Analog Reference** for the ADC.
It must NEVER be left floating.

You added **100nF capacitor (C3)** between:

- **AREF → GND**

This provides:

- Noise filtering
- Stable reference for ADC
- Clean readings from MQ-135 and RV1

Servo Motor Wiring (PB1 / OC1A)

Servos require:

- Fixed 20ms period
- 1ms to 2.4ms pulse width

ATmega32 Timer1 on PB1 is PERFECT for this.

Connections:

- PB1 → Servo Signal
- Servo +5V → Power supply
- Servo GND → MUST share same ground as MCU
- Servo internal control reads the pulse from OC1A

The code sets:

- 1000 μ s → Normal
- 1500 μ s → Middle
- 2000–2400 μ s → Fault rotation

This matches the required pulse width control.

Ground Reference Integrity

This is one of the MOST important parts of the circuit.

All grounds must be tied together:

- ATmega32 GND
- MQ-135 GND
- DHT22 GND
- Potentiometer GND
- Fan Transistor Emitter (Q1)
- Servo Motor GND
- Power Supply GND

If any ground is floating:

ADC values will be wrong

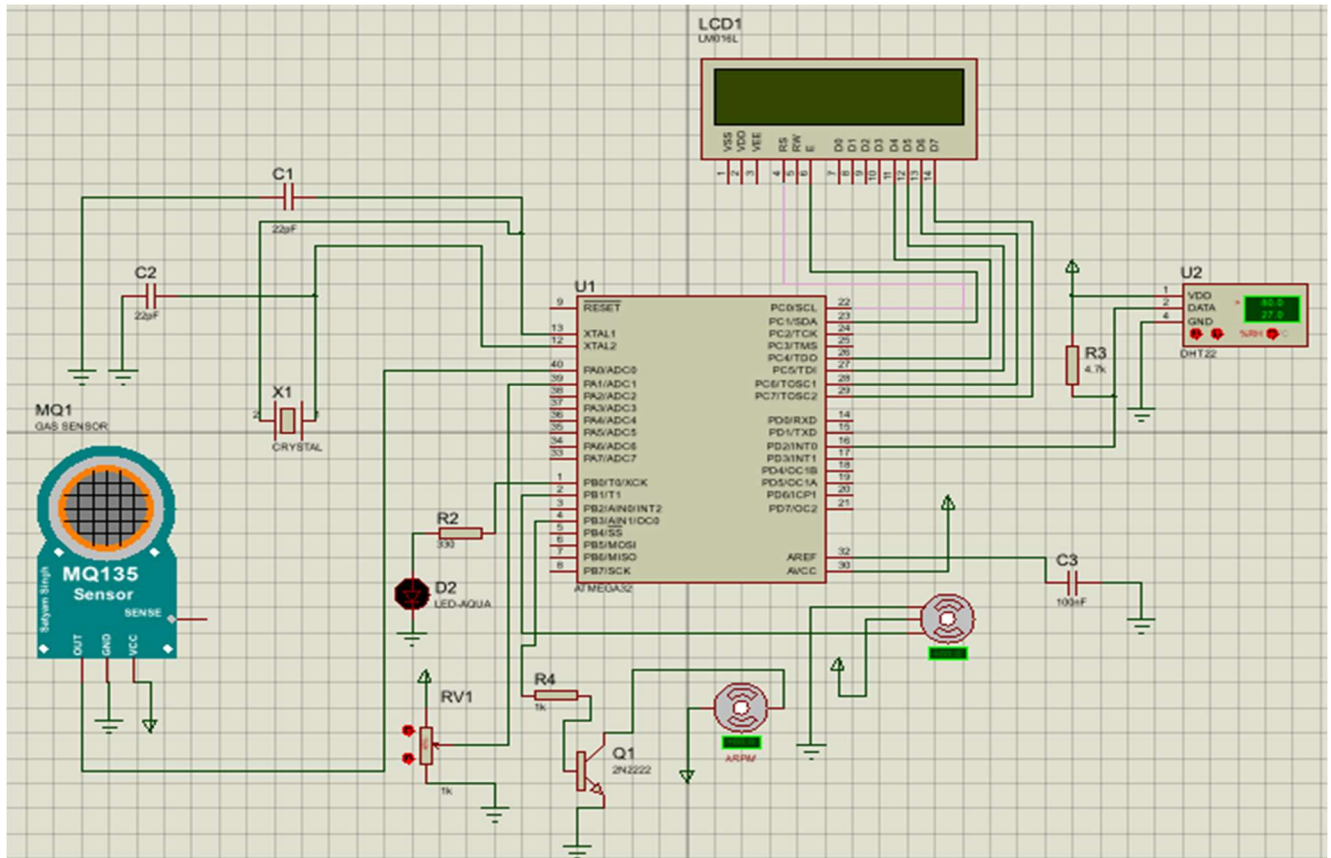
DHT22 will not respond

PWM will be unstable

Transistor will not switch

The circuit is showing correct ground merging, ensuring:

- Stable ADC
- Correct DHT22 communication
- Proper fan and servo operation



10. CODE (AVR ASSEMBLY)

```
.def V0 = r16
.def V1 = r17
.def V2 = r18
.def V3 = r19
.def T1 = r20
.def H1 = r21
.def MQH = r22
.def CSH = r23
.def TMP = r24
.def TMP2 = r25
```

```
.equ LCD_RS = PC0
.equ LCD_EN = PC1
.equ LCD_RW = PC2
```

```
.org 0x00
rjmp RESET
```

```
RESET:
    ldi V0, high(RAMEND)
    out SPH, V0
    ldi V0, low(RAMEND)
    out SPL, V0

    ldi V0, (1<<REFS0)
```

```

out ADMUX, V0
ldi V0, (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0)
out ADCSRA, V0

ldi V0, (1<<WGM00)|(1<<WGM01)|(1<<COM01)|(1<<CS01)
out TCCR0, V0
ldi V0, 0
out OCR0, V0

ldi V0, (1<<WGM11)|(1<<COM1A1)
out TCCR1A, V0
ldi V0, (1<<WGM13)|(1<<WGM12)|(1<<CS11)
out TCCR1B, V0
ldi V0, low(20000)
out ICR1L, V0
ldi V0, high(20000)
out ICR1H, V0
ldi V0, low(1500)
out OCR1AL, V0
ldi V0, high(1500)
out OCR1AH, V0

ldi V0, (1<<PB0)|(1<<PB1)|(1<<PB3)|(1<<PB5)
out DDRB, V0

ldi V0, 0xFF
out DDRC, V0
ldi V0, 0x00
out PORTC, V0

rcall DELAY_200MS
rcall DELAY_200MS

rcall LCD_INIT

rcall LCD_CLEAR
ldi ZH, high(STR_TITLE<<1)
ldi ZL, low(STR_TITLE<<1)
rcall LCD_PRINT_PGMEM
rcall DELAY_200MS
rcall DELAY_200MS
rcall DELAY_200MS

MAIN:
rcall READ_DHT22
mov T1, V1
mov H1, V2

ldi V0, 0
rcall ADC_READ
mov MQH, V2

ldi V0, 1
rcall ADC_READ
mov CSH, V2

cpi MQH, 1
brge GAS_FAULT

```

```

cpi CSH, 3
brge CUR_FAULT

cpi T1, 30
brge HIGH_TEMP

ldi V0, 80
out OCR0, V0
ldi V0, low(1000)
out OCR1AL, V0
clr V0
out OCR1AH, V0
cbi PORTB, PB0
cbi PORTB, PB1
rcall DISPLAY_NORMAL
rjmp LOOP_END

```

```

HIGH_TEMP:
ldi V0, 200
out OCR0, V0
ldi V0, low(2000)
out OCR1AL, V0
clr V0
out OCR1AH, V0
sbi PORTB, PB0
sbi PORTB, PB1
rcall DISPLAY_HIGH_TEMP
rjmp LOOP_END

```

```

GAS_FAULT:
ldi V0, 255
out OCR0, V0
ldi V0, low(2400)
out OCR1AL, V0
clr V0
out OCR1AH, V0
sbi PORTB, PB0
sbi PORTB, PB1
rcall DISPLAY_GAS_FAULT
rjmp LOOP_END

```

```

CUR_FAULT:
ldi V0, 255
out OCR0, V0
ldi V0, low(2400)
out OCR1AL, V0
clr V0
out OCR1AH, V0
sbi PORTB, PB0
sbi PORTB, PB1
rcall DISPLAY_CUR_FAULT
rjmp LOOP_END

```

```

LOOP_END:
rcall DELAY_200MS
rjmp MAIN

```

DISPLAY_NORMAL:

```
    rcall LCD_CLEAR

    ldi ZH, high(STR_T<<1)
    ldi ZL, low(STR_T<<1)
    rcall LCD_PRINT_PGMEM
    mov V0, T1
    rcall LCD_PRINT_BYTE_DEC
    ldi V0, 'C'
    rcall LCD_CHAR
    ldi V0, ''
    rcall LCD_CHAR
    ldi ZH, high(STR_H<<1)
    ldi ZL, low(STR_H<<1)
    rcall LCD_PRINT_PGMEM
    mov V0, H1
    rcall LCD_PRINT_BYTE_DEC
    ldi V0, '%'
    rcall LCD_CHAR

    rcall LCD_LINE2
    ldi ZH, high(STR_NORMAL<<1)
    ldi ZL, low(STR_NORMAL<<1)
    rcall LCD_PRINT_PGMEM
    ret
```

DISPLAY_HIGH_TEMP:

```
    rcall LCD_CLEAR
    ldi ZH, high(STR_T<<1)
    ldi ZL, low(STR_T<<1)
    rcall LCD_PRINT_PGMEM
    mov V0, T1
    rcall LCD_PRINT_BYTE_DEC
    ldi V0, 'C'
    rcall LCD_CHAR
    ldi ZH, high(STR_HIGH<<1)
    ldi ZL, low(STR_HIGH<<1)
    rcall LCD_PRINT_PGMEM
    rcall LCD_LINE2
    ldi ZH, high(STR_FAN_FULL<<1)
    ldi ZL, low(STR_FAN_FULL<<1)
    rcall LCD_PRINT_PGMEM
    ret
```

DISPLAY_GAS_FAULT:

```
    rcall LCD_CLEAR
    ldi ZH, high(STR_GAS<<1)
    ldi ZL, low(STR_GAS<<1)
    rcall LCD_PRINT_PGMEM
    rcall LCD_LINE2
    ldi ZH, high(STR_EVACUATE<<1)
    ldi ZL, low(STR_EVACUATE<<1)
    rcall LCD_PRINT_PGMEM
    ret
```

DISPLAY_CUR_FAULT:

```
    rcall LCD_CLEAR
    ldi ZH, high(STR_OVERCUR<<1)
```

```

ldi ZL, low(STR_OVERCUR<<1)
rcall LCD_PRINT_PGMEM
rcall LCD_LINE2
ldi ZH, high(STR_CHECKCIR<<1)
ldi ZL, low(STR_CHECKCIR<<1)
rcall LCD_PRINT_PGMEM
ret

```

LCD_INIT:

```

rcall DELAY_200MS
ldi V0, 0x03
rcall LCD_SEND_NIBBLE_CMD
rcall DELAY_1MS
rcall DELAY_1MS
rcall DELAY_1MS
rcall DELAY_1MS
rcall DELAY_1MS

```

```

ldi V0, 0x03
rcall LCD_SEND_NIBBLE_CMD
rcall DELAY_1MS
rcall DELAY_1MS

```

```

ldi V0, 0x03
rcall LCD_SEND_NIBBLE_CMD
rcall DELAY_1MS

```

```

ldi V0, 0x02
rcall LCD_SEND_NIBBLE_CMD
rcall DELAY_1MS

```

```

ldi V0, 0x28
rcall LCD_CMD

```

```

ldi V0, 0x0C
rcall LCD_CMD

```

```

ldi V0, 0x01
rcall LCD_CMD
rcall DELAY_1MS
rcall DELAY_1MS

```

```

ldi V0, 0x06
rcall LCD_CMD
ret

```

LCD_CMD:

```

mov TMP, V0
swap TMP
andi TMP, 0x0F
rcall LCD_SEND_NIBBLE_CMD
mov TMP, V0
andi TMP, 0x0F
rcall LCD_SEND_NIBBLE_CMD
rcall DELAY_1MS
ret

```

```

LCD_CHAR:
    mov TMP, V0
    swap TMP
    andi TMP, 0x0F
    rcall LCD_SEND_NIBBLE_DATA
    mov TMP, V0
    andi TMP, 0x0F
    rcall LCD_SEND_NIBBLE_DATA
    rcall DELAY_US_40
    ret

```

```

LCD_SEND_NIBBLE_CMD:

    mov V0, TMP
    swap V0
    andi V0, 0xF0
    out PORTC, V0
    ori V0, (1<<LCD_EN)
    out PORTC, V0
    rcall DELAY_US_40
    andi V0, ~(1<<LCD_EN)
    out PORTC, V0
    rcall DELAY_US_40
    ret

```

```

LCD_SEND_NIBBLE_DATA:
    mov V0, TMP
    swap V0
    andi V0, 0xF0
    ori V0, (1<<LCD_RS)
    out PORTC, V0
    ori V0, (1<<LCD_EN)
    out PORTC, V0
    rcall DELAY_US_40
    andi V0, ~(1<<LCD_EN)
    out PORTC, V0
    rcall DELAY_US_40
    ret

```

```

LCD_CLEAR:
    ldi V0, 0x01
    rcall LCD_CMD
    rcall DELAY_1MS
    rcall DELAY_1MS
    ret

```

```

LCD_LINE2:
    ldi V0, 0xC0
    rcall LCD_CM
    ret

```

```

LCD_PRINT_PGMEM:
LCD_PGMEM_LOOP:
    lpm V0, Z+
    cpi V0, 0
    breq LCD_PGMEM_DONE
    rcall LCD_CHAR

```

```

    rjmp LCD_PGMEM_LOOP
LCD_PGMEM_DONE:
    ret

LCD_PRINT_BYTE_DEC:
    mov TMP, V0

    clr TMP2

LCD_HUND:
    cpi TMP, 100
    brlt LCD_HUND_DONE
    subi TMP, 100
    inc TMP2
    rjmp LCD_HUND

LCD_HUND_DONE:
    tst TMP2
    breq LCD_TENS_START
    ldi V0, '0'
    add V0, TMP2
    rcall LCD_CHAR

LCD_TENS_START:
    clr TMP2

LCD_TENS:
    cpi TMP, 10
    brlt LCD_TENS_DONE
    subi TMP, 10
    inc TMP2
    rjmp LCD_TENS

LCD_TENS_DONE:
    tst TMP2
    breq LCD_UNITS
    ldi V0, '0'
    add V0, TMP2
    rcall LCD_CHAR

LCD_UNITS:
    ldi V0, '0'
    add V0, TMP
    rcall LCD_CHAR
    ret

ADC_READ:
    mov TMP, V0
    andi TMP, 0x07
    in TMP2, ADMUX
    andi TMP2, 0xF8
    or TMP2, TMP
    out ADMUX, TMP2
    sbi ADCSRA, ADSC
WAITADC:
    in TMP, ADCSRA
    sbrc TMP, ADSC
    rjmp WAITADC

```

```
in V1, ADCL
in V2, ADCH
ret
```

READ_DHT22:

```
sbi DDRD, PD2
cbi PORTD, PD2
rcall DELAY_1MS
sbi PORTD, PD2
cbi DDRD, PD2
rcall DELAY_US_40
```

```
rcall DHT_WL
rcall DHT_WH
```

```
clr V1
clr V2
```

```
ldi V3, 8
```

READ_TEMP_BIT:

```
rcall DHT_WH
rcall DELAY_US_40
lsl V1
sbis PIND, PD2
rjmp TEMP_SKIP
ori V1, 1
```

TEMP_SKIP:

```
rcall DHT_WL
dec V3
brne READ_TEMP_BIT
```

```
ldi V3, 8
```

READ_HUM_BIT:

```
rcall DHT_WH
rcall DELAY_US_40
lsl V2
sbis PIND, PD2
rjmp HUM_SKIP
ori V2, 1
```

HUM_SKIP:

```
rcall DHT_WL
dec V3
brne READ_HUM_BIT
ret
```

DHT_WH:

```
sbis PIND, PD2
rjmp DHT_WH
ret
```

DHT_WL:

```
sbic PIND, PD2
rjmp DHT_WL
ret
```

DELAY_1MS:

```
ldi V0, 250
```

```

MS1:
    ldi V3, 4
MS2:
    dec V3
    brne MS2
    dec V0
    brne MS1
    ret

DELAY_US_40:
    ldi V0, 40
U40:
    dec V0
    brne U40
    ret

DELAY_200MS:
    ldi V0, 200
D200:
    rcall DELAY_1MS
    dec V0
    brne D200
    ret

STR_TITLE: .db " HVAC SYSTEM ", 0
STR_T: .db "T:", 0
STR_H: .db " H:", 0
STR_NORMAL: .db "Status: NORMAL ", 0
STR_HIGH: .db "C HIGH! ", 0
STR_FAN_FULL: .db "Fan: FULL SPEED ", 0
STR_GAS: .db "GAS DETECTED! ", 0
STR_EVACUATE: .db "EVACUATE NOW! ", 0
STR_OVERCUR: .db "OVERCURRENT! ", 0
STR_CHECKCIR: .db "Check Circuit ", 0

```

10. Cost Analysis:

COMPONENTS	QUANTITY	PRICE(PKR)	TOTAL
ATmega32	1	850	850
DHT22	1	450	450
MQ-135	1	400	400
ACS712	1	450	450
SG90 Servo	1	500	500
Relay Module	1	300	300
IRFZ44N MOSFET	1	450	450
Fan 12V	1	300	300

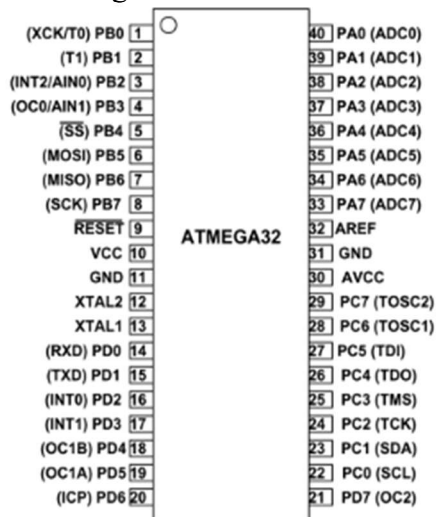
Crystal 16 MHz	1	400	400
Jumper wires	pack	300	300
TOTAL COST	—	—	4400

11. United Nations Social Development Goals SDG's:

GOAL NO	GOAL	HOW
3	Good health and well-being	Monitors CO ₂ /gas levels and temperature to maintain safe indoor air quality
7	Affordable and clean energy	Fan and servo only activate when needed, energy is not wasted running at full speed constantly
9	Industry, Innovation and Infrastructure	Embedded automation replacing manual HVAC control, directly applicable to smart building infrastructure
11	Sustainable cities and communities	Smart environmental control contributes to healthier, more efficient buildings in urban settings
13	Climate action	Optimised energy use in buildings reduces overall carbon footprint

12. APPENDICES

Pin configuration of a ATMEGA32:



LCD Interfacing:

